CS-233 — Introduction to Machine Learning

Course Project — Milestone 1 Report

Sara Iraqi (391295) Jad Chahed Ouazzani (391350) Riad Elhariry (399144)

1. Introduction

This milestone addresses the prediction of a user's video-game addiction from 13 features describing gaming habits and mental-health status, using a subset of the Gaming and Mental Health Dataset (1600 training / 400 test samples). Two tasks are defined on the same inputs: a three-class classification of the addiction level (*Low*, *Medium*, *High*, encoded as $\{0, 1, 2\}$) and a regression of a continuous addiction score in $[0, 10]$. We implement three algorithms from the lectures: *linear regression* (closed-form), multinomial *logistic regression* (batch gradient descent) and *k-NN* (both tasks). Hyperparameters are selected on a held-out validation set; final test-set numbers are obtained after retraining on all 1600 samples.

2. Method

Data preparation. We extract a random 80/20 split from the 1600 training samples (seed = 100), giving 1280 training and 320 validation points. Standardisation statistics are computed on the training split only, so no label information leaks across the split. A bias column of ones is prepended inside each linear method. The class distribution is strongly imbalanced: 68.9%/27.9%/3.2% (*Low* / *Medium* / *High*).

Linear regression. Closed-form least squares, $\mathbf{w}^\star = (X^\top X)^{-1} X^\top \mathbf{y}$, on the bias-augmented design matrix. No hyperparameter.

Logistic regression. Multinomial softmax $\hat{y}^{(k)}(\mathbf{x}) = \exp(\mathbf{w}_{(k)}^\top \mathbf{x}) / \sum_j \exp(\mathbf{w}_{(j)}^\top \mathbf{x})$ with one-hot targets, trained by batch gradient descent on the cross-entropy loss. The softmax is stabilised by subtracting $\max_j z_j$ from each logit row before exponentiating; without this, learning rates above $\sim 5 \cdot 10^{-2}$ produce `NaN`s. Hyperparameters: learning rate η and `max_iters`.

k-NN. Non-parametric: training stores the data. For each test point, distances to all training points are computed and the k nearest neighbours are used to predict by majority vote (classification) or mean (regression). We implement both the Euclidean distance $d_{L_2}(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_d (a_d - b_d)^2}$ and the Manhattan distance $d_{L_1}(\mathbf{a}, \mathbf{b}) = \sum_d |a_d - b_d|$. Hyperparameters: k and the distance.

3. Experiments & Results

Hyperparameter selection. Figures 1–3 show the validation-set sweeps. For k-NN classification, accuracy peaks at $k = 10$ (val. acc. 79.69%) while macro-F1 peaks at $k = 3$ (0.620); we retain $k = 10$ as the accuracy-optimal choice and return to this in the Discussion. For k-NN regression, MSE follows the classic bias-variance U-shape, with $k = 10$ and $k = 15$ essentially tied (val. MSE 1.990 vs 1.998); we retain $k = 15$. For logistic regression, accuracy is flat across a broad plateau $\eta \in [5 \cdot 10^{-4}, 5 \cdot 10^{-3}]$ (val. acc. 86.25–86.56%), degrades at $\eta = 10^{-2}$ and collapses at $\eta = 5 \cdot 10^{-2}$; the convergence plot shows accuracy plateaus from $\sim$200 iterations. We retain $\eta = 5 \cdot 10^{-3}$, `max_iters`$= 500$.

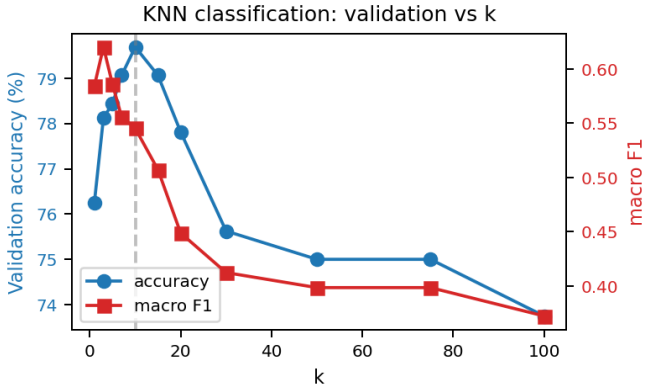


Figure 1: k-NN classif. sweep; best $k = 10$.

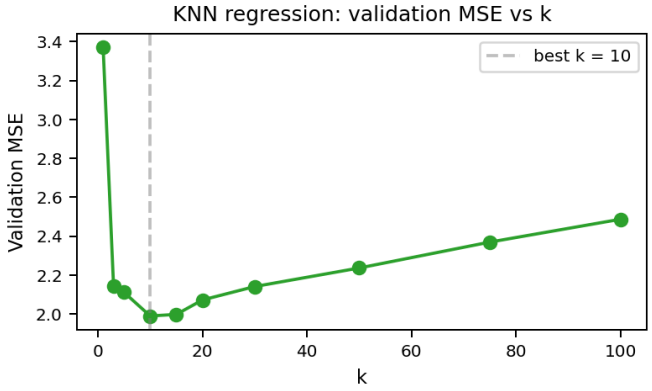


Figure 2: k-NN reg. sweep; U-shape, best $k \approx 10$–15.

Distance-function comparison. As Lecture 5 notes that the distance choice affects k-NN's behaviour, we compare L_1 and L_2 on the test set (Fig. 4). For regression, L_2 uniformly dominates L_1 across all tested k (1.832 vs 2.015 MSE at $k = 15$). For classification, L_1 beats L_2 only at very small k (82.5% vs 79.0% at $k = 5$), while L_2 is better for $k \geq 7$ and its macro-F1 is systematically higher. We therefore keep L_2 as the final choice.

Final test performance. Table 1 summarises the test-set numbers after retraining on all 1600 samples. All four configurations beat the baselines suggested in the project statement (80%, 70%, 1.1 MSE, 2.1 MSE).

4. Discussion & Conclusion

On both tasks the parametric linear methods clearly outperform k-NN. After standardisation the 13 features appear approximately linearly separable, which plays to the strength of a linear decision boundary; conversely, a 13-dimensional Euclidean metric becomes less discriminative (*curse of dimensionality*, Lec. 5) and k-NN cannot weight features by relevance.

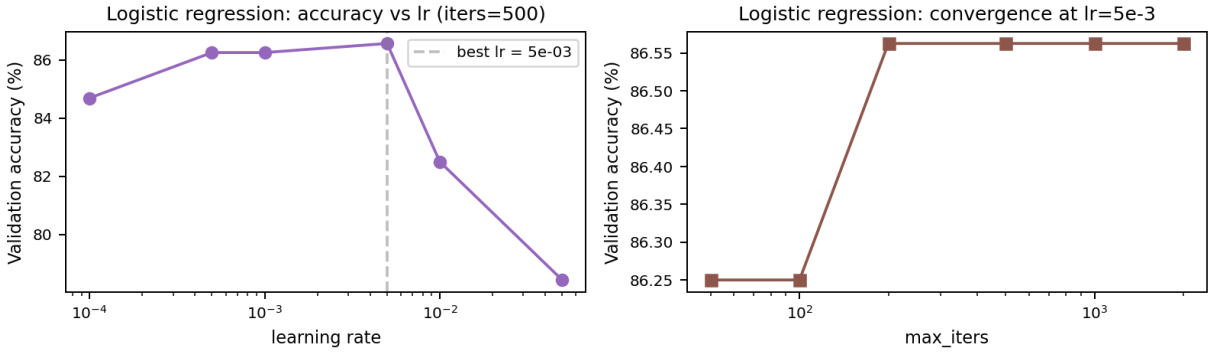


Figure 3: Logistic regression sweeps. **Left:** validation accuracy vs. learning rate (iters = 500). **Right:** convergence at $\eta = 5{\cdot}10^{-3}$; accuracy plateaus around 86.6% from ∼200 iterations.

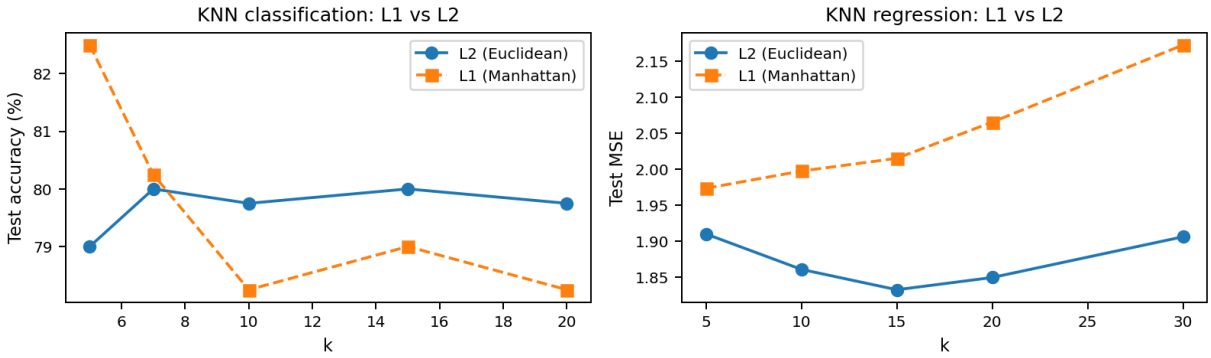


Figure 4: L_1 vs L_2 distance on the test set. **Left:** classification accuracy; L_1 beats L_2 only at very small k. **Right:** regression MSE; L_2 dominates uniformly.

Method (task, hyperparameters)	**Train acc/MSE**	**Train F1**	**Test acc/MSE**	**Test F1**
Logistic reg. ($\eta = 5{\cdot}10^{-3}$, iters = 500)	86.00%	0.794	**88.75%**	**0.863**
k-NN classif. ($k = 10$, L_2)	81.06%	0.643	79.75%	0.510
Linear regression	0.986	—	**0.994**	—
k-NN reg. ($k = 15$, L_2)	1.652	—	1.832	—

Table 1: Final test-set results after retraining with the selected hyperparameters. Best per task in **bold**.

The gap between accuracy and macro-F1 for k-NN (79.75% vs 0.510) is directly explained by the class imbalance. A per-class breakdown on the test set quantifies this: for k-NN ($k = 10$) we obtain F1 = 0.89/0.50/0.14 on *Low* / *Medium* / *High* with recalls 0.99/0.37/0.08; for logistic regression F1 = 0.93/0.77/0.89 with recalls 0.95/0.71/0.92. Among the 13 *High* test samples, k-NN correctly identifies only 1, while logistic regression identifies 12. This makes macro-F1 a much more honest metric than accuracy on this dataset, as warned in the project statement. The macro-F1 peaking at $k = 3$ (Fig. 1) fits the same diagnosis: smaller neighbourhoods are more likely to contain a minority-class example, so recall on rare classes improves at the cost of stability.

For regression, linear regression wins by a factor of roughly two in MSE (0.994 vs 1.832), consistent with a signal well-explained by a linear combination of the standardised features. k-NN regression shows the classic bias-variance trade-off: $k = 1$ overfits (val. MSE 3.37) and large k underfits (MSE 2.49 at $k = 100$). The L_1 vs L_2 nuance is worth noting: for regression, averaging over geometrically-close neighbours is what matters, so L_2's isotropic geometry wins uniformly; for classification, L_1 at $k = 5$ gives a higher accuracy but a worse macro-F1, so under a balanced-performance criterion L_2 is still preferable.

In terms of runtime, linear methods are essentially free: linear regression fits in < 1 ms (single matrix solve), logistic regression trains in ∼100 ms for 500 iterations; both predict on 400 test points in < 1 ms. k-NN has no training cost but pays it at test time: the 400×1600 distance matrix takes ∼50–100 ms and scales linearly with N. Overall, linear parametric methods dominate on this dataset. For Milestone 2 we plan to test multi-layer perceptrons to see whether a non-linear model can extract signal beyond what the linear methods already capture.